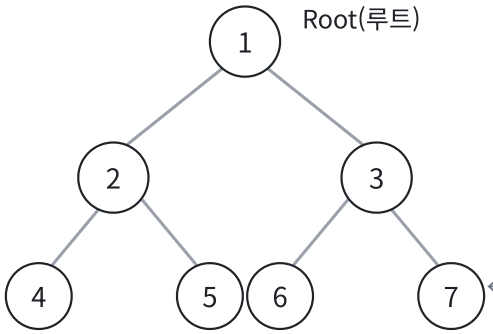
CSED233 · 데이터구조 · 포스텍

정리본 5. Trees

트리 용어 · Depth vs Height · 이진 트리 3가지 특수형 · 배열/연결 구현.

1. Tree(트리) — 계층적 자료구조

Tree(트리): 노드와 간선으로 구성된 연결되고 사이클이 없는 계층적 자료구조. 비어 있지 않은 트리에는 루트가 정확히 하나 존재하며, 루트는 부모가 없고 나머지 모든 노드는 정확히 하나의 부모를 가진다. 루트에서 각 노드까지의 경로는 유일하다.



용어
· Parent/Child: 위/아래 노드 (1은 2의 부모)
· Siblings(형제): 같은 부모 (2와 3)
· Path(경로): 루트→노드의 길 ⟨1,2,4⟩
· Edge(간선): 노드를 잇는 선

← Leaf(리프): 자식 없는 노드 (4,5,6,7)

Depth: Root에서의 거리 (Root = 0, 아래로 증가)
Height: Leaf까지의 최장 거리 (Leaf = 0, 위로 증가)
이 트리: Depth(4)=2, Height(1)=2 → 트리 높이 = 2

같은 트리를 두 기준으로 본다: Depth는 위에서부터, Height는 아래에서부터

Depth와 Height를 헷갈리지 말 것

- **Depth(깊이)**: 루트에서 해당 노드까지의 간선 수. Root=0.
- **Height(높이)**: 해당 노드에서 가장 먼 리프까지의 간선 수. Leaf=0.
- **트리의 높이 = Root의 Height.**

2. Binary Tree(이진 트리)와 특수형

Binary Tree(이진 트리): 각 노드가 최대 2개의 자식을 가지는 트리. Left/Right가 명시적으로 구분된다(일반 트리와 다른 점).

```
struct TreeNode {
  int val;
  TreeNode* left;  // 왼쪽 자식
  TreeNode* right; // 오른쪽 자식
};
```

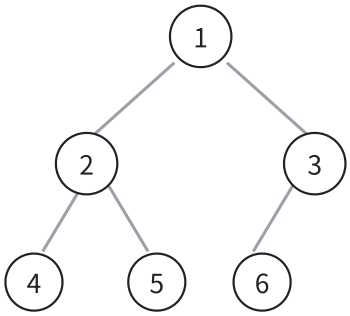
2.1 Complete Binary Tree(완전 이진 트리)

마지막 레벨을 제외한 모든 레벨이 완전히 채워지고, 마지막 레벨은 왼쪽부터 차례로 채운 트리. 순서대로 채워지므로 **배열로 구현 가능**하다(포인터 불필요).

배열 인덱스 관계식 (1-indexed, 0번 칸은 비움)

$$\text{parent}(i) = \lfloor i/2 \rfloor, \quad \text{left}(i) = 2i, \quad \text{right}(i) = 2i + 1$$

검증: 노드 4의 부모 = 배열[⌊4/2⌋] = 배열[2] = 2 ✓ · 노드 2의 왼쪽 = 배열[4] = 4 ✓



레벨 1까지 완전 · 마지막 레벨은 왼쪽부터 (3의 오른쪽 자리는 비어 있음)

배열 표현: 노드 번호 = 인덱스

—	1	2	3	4	5	6
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Complete Binary Tree → 배열 그대로 대응

2.2 Proper(0 또는 2개 자식) / Full(모든 레벨 충족)

- **Proper(정식 이진 트리)**: 각 노드가 자식을 0개 또는 2개 가짐 (1개는 금지).
- **Full Binary Tree(포화 이진 트리)**: 모든 레벨이 완전히 채워진 트리. 높이 h이면 노드 수는:

$$n = 2^{h+1} - 1$$

예: 높이 2 → $2^3 - 1 = 7$개 노드.



왼쪽은 모든 내부 노드가 자식 2개, 오른쪽은 위반

2.3 Balanced Binary Tree(균형 이진 트리)

여기서는 AVL 방식의 균형 조건을 사용한다: 모든 노드에서 왼쪽/오른쪽 서브트리의 높이 차이 ≤ 1인 트리. 높이가 O(log n)으로 억제된다. BST의 순서 조건까지 만족하면 검색이 O(log n)으로 보장된다 (일반 이진 트리의 검색은 순서 조건이 없어 O(n)일 수 있음). 비균형(한쪽으로만 긴) 트리는 높이가 O(n)까지 커져 성능이 저하된다 — 그래서 AVL/Red-Black Tree가 자동 균형을 잡는다(강의 10).

3. 구현: 배열 vs 연결

	배열 기반	Linked 기반
적합한 형태	Complete에 특히 효율적	모든 형태의 Binary Tree
관계 표현	인덱스 계산 (⌊i/2⌋, 2i, 2i+1)	left/right 포인터
위치를 아는 노드 접근	인덱스로 O(1)	포인터로 O(1)
값 검색(순서 조건 없음)	최악 O(n)	최악 O(n)
장점	포인터 불필요·연속 저장	임의 형태에 유연
단점	희소한 형태에서 빈 공간 낭비	포인터 저장 공간 필요

배열의 Random Access O(1)은 인덱스를 안다는 전제이며 Linked도 포인터를 알면 O(1), 목표 값만 알고 순서 조건 없이 검색하면 두 구현 모두 최악 O(n)이 될 수 있다.

3.1 배열 기반 예시

```
int tree[MAX];  // 1-indexed (0번 인덱스는 안 씀)
tree[1] = 10
tree[2] = 5,  tree[3] = 15
tree[4] = 3,  tree[5] = 7,  tree[6] = 12,  tree[7] = 20
//        10(1)
//       /    \
//     5(2)    15(3)
//    /   \    /   \
//  3(4) 7(5) 12(6) 20(7)
```

3.2 Linked 기반 실전 예제

```
#include <iostream>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left  = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left  = new TreeNode(4);
    root->left->right = new TreeNode(5);
    //        1
    //       / \
    //      2   3
    //     / \
    //    4   5
    return 0;
}
```

이 트리의 순회(강의 6에서 자세히): Preorder 1 2 4 5 3 · Inorder 4 2 5 1 3 · Postorder 4 5 2 3 1

4. 핵심 정리

- 트리 연산의 시간복잡도는 연산 종류와 트리의 성질에 따라 달라진다. BST 검색·삽입·삭제는 한 경로를 따라 진행하므로 O(h). 균형 BST는 O(log n), 한쪽으로 치우친 BST는 최악 O(n). 일반 Binary Tree의 값 검색은 순서 조건이 없으면 균형이어도 최악 O(n). 전체 트리 순회는 항상 Θ(n). 비균형 트리는 최악의 경우 높이가 Θ(n)까지 커질 수 있다.
- Depth는 위에서(Root=0), Height는 아래에서(Leaf=0) 센다.
- 특수형 4가지:
  - Complete — 마지막 레벨을 제외한 모든 레벨이 채워지고 마지막 레벨은 왼쪽부터 채워진 트리, 배열 구현에 효율적.
  - Proper — 모든 노드가 자식을 0개 또는 2개 가지는 트리.
  - Full(이 노트의 포화 정의, Perfect) — 모든 레벨이 완전히 채워진 트리.
  - Balanced(여기서는 AVL 조건) — 모든 노드의 왼쪽·오른쪽 서브트리 높이 차가 1 이하인 트리.
- 배열 구현은 Complete에 특히 효율적 (다른 형태도 가능하나 공간 낭비 큼), Linked 구현은 모든 형태 가능.

5. Quiz

Q1. Depth(깊이)와 Height(높이)의 차이는?

> 힌트

세는 방향이 반대다: 하나는 Root에서, 하나는 Leaf에서.

> 정답

개념은 힌트 참고.

정답: Depth는 위에서 아래로(Root=0), Height는 아래에서 위로(Leaf=0) 센다. 트리의 높이는 Root의 Height.

Q2. Complete Binary Tree가 배열로 구현되는 이유는?

> 힌트

빈틈없이 순서대로 채워지면 위치를 "계산"할 수 있다.

정답

개념은 힌트 참고.

정답: 순서대로 채워져 있어 인덱스 계산(⌊i/2⌋, 2i, 2i+1)으로 부모-자식 관계를 바로 알 수 있어 포인터가 필요 없기 때문. 메모리도 효율적.

Q3. 배열 구현에서 인덱스 10에 있는 노드의 부모는?

힌트

parent(i) = ⌊i/2⌋.

정답

개념은 힌트 참고.

① 공식 적용: ⌊10/2⌋ = 5.

② 배열[5]에 있는 노드가 부모다.

정답: 인덱스 5의 노드 (⌊10/2⌋ = 5).

Q4. Full Binary Tree와 Complete Binary Tree의 차이는?

힌트

Full은 "모든 레벨 충족" 규칙, Complete는 "채우는 순서" 규칙, Proper은 "자식 개수" 규칙.

정답

개념은 힌트 참고.

정답: Full은 모든 레벨이 가득 찬 트리, Complete는 레벨 순서대로 빈틈없이 채운 트리. (Full은 Complete이지만 Complete이라고 Full은 아니다. 자식 1개가 금지된 트리는 Proper.)

Q5. 비균형 Binary Tree의 문제점은?

힌트

연산 비용은 높이 h에 비례한다. 한쪽으로 길어지면 h는 간선 수로 n−1까지 커진다.

정답

개념은 힌트 참고.

① 높이: n개의 노드가 한쪽으로만 연결되면 간선 수 기준 높이는 $h=n-1=\Theta(n)$이다.

② 연산 비용: BST는 검색·삽입 시 루트부터 한 경로를 따라 이동하므로 비용이 높이에 비례한다. 균형 BST에서는 O(log n)이지만 한쪽으로 치우친 최악의 경우 O(n)이 된다.

③ 해결: AVL Tree나 Red-Black Tree처럼 높이를 O(log n)으로 유지하는 자기 균형 BST를 사용한다.

정답: 비균형 BST는 최악의 경우 높이가 n−1까지 증가하여 검색·삽입이 O(n)으로 악화된다. 주의: BST 순서 조건이 없는 일반 이진 트리는 균형이어도 값 검색이 최악 O(n)일 수 있다.